

AI Programming Foundations Project: Module Summary

Grant Collings

AI Mastery Capstone - Project 2

Overview

This project builds a reproducible data workflow using public Cincinnati Fire/EMS CAD incident data. The purpose of the workflow is to practice the full early-stage data process: loading a real CSV file, inspecting structure and quality, cleaning selected fields, creating useful analysis features, producing exploratory summaries, and communicating findings through visualizations and interpretation. The project does not perform machine learning or operational decision support. It is limited to academic data workflow practice.

Dataset Description

The dataset used in this project is `cincinnati_fire_incidents_2025.csv`, a public Cincinnati Fire/EMS CAD incident dataset (City of Cincinnati, n.d.). The local file contains 97,881 rows and 17 columns. The fields include incident location information, incident creation time, dispatch time, arrival time, closure time, incident type, disposition, beat, neighborhood, and community council neighborhood.

This dataset was selected because it is public, tabular, large enough for meaningful exploration, and closely related to the type of municipal incident data that may be useful in later public-safety analytics projects. The dataset does not include patient data, departmental data, or PHI. Because the file is a real municipal export, it also includes realistic data-quality issues such as missing values, text fields that need standardization, and timestamp columns stored as text.

Workflow Description

The workflow begins by importing pandas, NumPy, and Matplotlib. The CSV file is then loaded into a pandas DataFrame. The notebook displays the dataset shape, column names, data types, sample rows, descriptive statistics, and missing-value counts.

The first cleaning function, `clean_text_columns`, strips whitespace from text columns and standardizes blank text values as missing values. This supports cleaner grouping and counting during exploratory analysis. The second cleaning function, `convert_time_columns`, converts the incident time fields from text into datetime values. This step is important because the original CSV stores timestamp values as strings, which limits time-based analysis until conversion is complete.

After cleaning, the notebook creates three analysis features:

1. `incident_hour`
2. `incident_day_name`
3. `dispatch_to_arrival_minutes`

The `dispatch_to_arrival_minutes` field is calculated from primary unit dispatch and arrival timestamps. This makes it possible to explore one simplified response-time interval, while still recognizing that this interval is not a complete measure of operational performance.

The notebook also includes an exploratory data analysis function, `summarize_incident_data`, which prints the dataset shape, top incident types, top dispositions, top neighborhoods, and summary statistics for dispatch-to-arrival time.

Key Decisions and Assumptions

One key decision was to keep the original public CSV file in the repository because the file size is manageable and it helps reviewers rerun the notebook without needing to download the data separately. Another decision was to preserve the original column names rather than renaming every field. This keeps the notebook connected to the source export while still allowing selected derived fields to be added.

The workflow uses INCIDENT_TYPE_ID as the main incident-type field because INCIDENT_TYPE_DESC, CFD_INCIDENT_TYPE, and CFD_INCIDENT_TYPE_GROUP contain substantial missing values in this local file. The notebook therefore avoids over-relying on columns that are mostly empty. This is a practical data-quality decision rather than a claim that those fields are unimportant.

The response-time calculation assumes that DISPATCH_TIME_PRIMARY_UNIT and ARRIVAL_TIME_PRIMARY_UNIT represent comparable timestamps for the primary responding unit. Missing, invalid, negative, or extreme intervals are handled cautiously during visualization by filtering the histogram to values between 0 and 60 minutes.

Reproducible workflow design is important because reviewers and future users need to rerun the analysis and understand each step. Danchev (2022) emphasizes reproducible data science practices using Python-based workflows, which supports this project's use of a structured notebook, local data file, and requirements file. Wickham (2014) also supports the importance of organizing and cleaning tabular data so that it can be manipulated, visualized, and interpreted more reliably.

\### Results and Interpretation

The dataset contains 97,881 incident records and 17 columns. Initial inspection showed that most core fields were populated, including location, agency, creation time, event number, and coordinates. Several descriptive fields had substantial missingness. In particular, `INCIDENT\_TYPE\_DESC`, `CFD\_INCIDENT\_TYPE`, and `CFD\_INCIDENT\_TYPE\_GROUP` were missing in most records. This supported the decision to use the more complete `INCIDENT\_TYPE\_ID` field for incident-category exploration.

The most common incident type IDs included EMS, fire-alarm, person-down, accident, and informational categories. Common dispositions included transport, investigation, cancellation, patient refusal, transfer, and release outcomes. Neighborhood counts were highest in Westwood, Downtown, Avondale, East Price Hill, West Price Hill, Over-the-Rhine, West End, Walnut Hills, CUF, and College Hill.

\\### Figure 1: Top Incident Type IDs

Figure 1 shows that EMS was the largest individual incident category, with 4,792 records, followed by `=FALARM` with 4,278 records. The remaining leading categories included `PERDWN - 32D1 UNKNOWN`, `ACCI - (C) =`, and `=INFOF`. The figure therefore demonstrates that the dataset contains a varied mixture of EMS, fire-alarm, person-down, accident, informational, and other calls rather than one overwhelmingly dominant category.

This finding supports the use of incident-type grouping for descriptive analysis, but it should be interpreted cautiously. `INCIDENT\_TYPE\_ID` contains operational coding labels rather than fully standardized plain-language categories. In addition, the more descriptive incident-type fields were largely missing, limiting the amount of detail available for validating or consolidating the codes.

\\### Figure 2: Incident Counts by Hour of Day

Figure 2 shows that incident demand was not distributed evenly throughout the day. Counts were lower during the overnight and early-morning hours, increased during the daytime, and were highest during the afternoon and evening. This reveals a clear time-of-day pattern in the recorded incidents.

The pattern may be useful for identifying periods of higher recorded activity, but it does not establish why call volume changes by hour or show whether staffing was sufficient. Population activity, commuting, business hours, weather, incident severity, reporting behavior, and other factors could contribute to the observed distribution. The chart is therefore an exploratory description of call timing rather than an operational staffing recommendation.

\\### Figure 3: Dispatch-to-Arrival Time Distribution

Figure 3 shows that dispatch-to-arrival times were concentrated in the low single digits, with a median of approximately 4.4 minutes. Most observations were grouped toward the lower end of the displayed range, while fewer incidents had substantially longer intervals. This right-skewed pattern means the median is more representative of a typical observation than the mean alone because longer-duration records can pull the mean upward.

The histogram was limited to intervals between 0 and 60 minutes to make the main distribution readable and reduce the visual influence of negative, invalid, or extreme values. This filtering improves interpretation of the chart but does not prove that every excluded record was erroneous. Some extreme intervals may reflect unusual incidents, staging, documentation practices, delayed timestamp entry, or other operational circumstances requiring further investigation.

\### Overall Interpretation

Together, the three figures show that the dataset contains varied incident categories, a recognizable time-of-day pattern, and a right-skewed dispatch-to-arrival distribution centered in the low single digits. They demonstrate how cleaned tabular data and derived time features can produce meaningful exploratory findings.

Neighborhood totals also show that recorded incident volume was not evenly distributed across Cincinnati. However, raw counts do not account for population, daytime population, geography, call density, hazards, resource placement, or reporting practices. Population-adjusted rates and additional contextual data would be required before making fair comparisons among neighborhoods.

These findings describe only the records in this dataset. They should not be interpreted as causal conclusions, personnel evaluations, performance standards, or recommendations for operational resource allocation.

Responsible Practice

This dataset is public municipal incident data, but responsible use still matters. Incident records may reflect community conditions, reporting practices, dispatch coding, resource availability, and system documentation habits. High incident volume in a neighborhood should not be interpreted as a simple statement about the people who live there. It may reflect many overlapping factors, including population density, service demand, geography, infrastructure, and public reporting patterns.

The workflow also avoids making operational, clinical, or performance claims from the data. Dispatch-to-arrival time is only one simplified interval and does not capture the full incident timeline, resource constraints, travel conditions, call severity, staging, scene safety, or patient outcome. Because of that, the notebook treats response-time exploration as a data workflow exercise rather than a performance evaluation.

A major data-quality limitation is missingness. Several descriptive incident-type fields are mostly missing in this local file. This could affect interpretation if those fields were expected to contain more detailed classification information. The workflow responds by documenting missing values and using more complete fields for analysis.

Reproducibility

The project is designed so the notebook can be rerun from top to bottom. The repository includes the local CSV dataset, the main notebook, a README with run instructions, and a requirements.txt file generated from the Python environment. Git is used to track project progress, including separate commits for the README, notebook, and requirements file. The repository also includes a branch beyond main, which supports the required Git workflow.

The notebook was executed successfully using Jupyter through Python. The executed notebook includes code outputs and visualizations, which helps connect the written interpretation to actual results produced by the workflow.

References

City of Cincinnati. (n.d.). Cincinnati Fire Incidents (CAD) including EMS: ALS/BLS. Cincinnati Open Data. <https://data.cincinnati-oh.gov/Safety/Cincinnati-Fire-Incidents-CAD-including-EMS-ALS-BL/vnsz-a3wp>

Danchev, V. (2022). Reproducible data science with Python: An open learning resource. *Journal of Open Source Education*, 5(56), 156. <https://doi.org/10.21105/jose.00156>

Wickham, H. (2014). Tidy data. *Journal of Statistical Software*, 59(10), 1-23. <https://doi.org/10.18637/jss.v059.i10>